

KukaLog

Plataforma de Gestión de Incidencias Técnicas

Robots KUKA KRC2

Ford Motor Company
Body Construction · Planta de Valencia

Documento generado: 23 de mayo de 2026

Autor:	Miguel Serra Ferrando
Departamento:	Body Construction Engineering
Tutor:	Javi Llopis

Índice

Parte I — Executive Pitch	2
1. El Problema	2
1.1. Problemas del sistema actual	2
1.2. Datos clave	2
2. La Solución: KukaLog	2
2.1. Capacidades principales	2
3. Propuesta de Valor	3
4. Roadmap	3
5. Coste e Infraestructura	3
6. Usuarios Objetivo	3
Parte II — Documentación Técnica	5
7. Arquitectura del Sistema	5
7.1. Flujo de datos	5
8. Stack Tecnológico	5
9. Modelo de Datos	5
9.1. Categorías de incidencia	6
10.API Reference	6
10.1. Filtros disponibles (query params)	7
10.2. Ejemplo de petición	7
10.3. Ejemplo de respuesta (stats)	7
11.Frontend — Páginas	7
11.1. Diseño visual	7
12.Deployment	8
12.1. Docker Compose (local / desarrollo)	8
12.2. Comandos	8
12.3. Cloud Run (producción)	8
Parte III — Implementaciones Futuras	9
13.Fase 2: Multi-usuario Corporativo	9
13.1. Autenticación SSO (Azure AD)	9
13.2. Migración a PostgreSQL	9
13.3. Upload de archivos multimedia	9

14.Fase 3: Integración con Agentes IA	9
14.1. Kratos consulta KukaLog	9
14.2. Registro automático desde lenguaje natural	10
14.3. Analytics avanzado	10
15.Fase 4: Multi-planta	11
16.Equipo y Créditos	12

Parte I — Executive Pitch

El Problema

En la planta de Ford Valencia, los ingenieros y operarios de Body Construction gestionan las incidencias técnicas de robots KUKA KRC2 mediante un **chat informal de Microsoft Teams**.

Cuando ocurre un fallo en un robot, el operario escribe en el chat:

- Qué ha ocurrido
- Qué robot ha fallado
- Qué código de error apareció
- Cómo se solucionó
- Imágenes o notas adicionales

Problemas del sistema actual

Problema	Impacto
Información desordenada	Imposibilidad de encontrar soluciones previas
Sin estructura	No se puede filtrar por robot, error, fecha, línea
Búsquedas ineficaces	Un ingeniero puede tardar 30–60 min en encontrar un caso similar
Soluciones repetidas	El mismo problema se resuelve de cero múltiples veces
Información que se pierde	Mensajes antiguos quedan enterrados en el historial
Sin trazabilidad	No hay métricas, no se sabe qué robots fallan más
Sin transferencia de conocimiento	Cuando un técnico se va, su experiencia se pierde

Datos clave

Basándose en el histórico del chat de Teams, se han identificado:

- Más de **40 tipos de incidencias** recurrentes documentadas
- Fallos repetitivos como *Green Goo*, fallos KSD, KPS, SSD, ventiladores
- Robots con historial extenso de intervenciones (ej: robots de líneas 6X, 6Y, 8XY)
- Procedimientos críticos no documentados formalmente (UDMA BIOS, \$RebootDSE, restauración SSD)

La Solución: KukaLog

KukaLog es una plataforma web interna diseñada para reemplazar el sistema informal de chat por una herramienta estructurada, rápida y profesional.

Capacidades principales

1. **Registro estructurado** — Formulario profesional para documentar incidencias con todos los campos técnicos necesarios
2. **Búsqueda inteligente** — Búsqueda full-text con filtros avanzados por robot, error, categoría, línea, fecha, prioridad
3. **Consulta de histórico** — Acceso inmediato a soluciones previas de problemas similares
4. **Dashboard analítico** — Métricas de planta: errores frecuentes, robots problemáticos, tiempos de parada

5. Trazabilidad — Historial completo de cada robot con todas sus incidencias

Propuesta de Valor

Métrica	Antes (Chat Teams)	Con KukaLog
Buscar solución previa	30–60 minutos	5–30 segundos
Documentar incidencia	Texto libre desordenado	Formulario estructurado
Transferencia entre turnos	Verbal / mensajes perdidos	Registro persistente consultable
Métricas de planta	Inexistentes	Dashboard en tiempo real
Trazabilidad por robot	Manual	Automática
Búsqueda por código error	Scroll infinito en chat	Un clic con filtro
Onboarding técnicos nuevos	Semanas de experiencia	Acceso inmediato al histórico

Roadmap

Visión estratégica en 3 fases

- Fase 1 — MVP (completado)**
Aplicación web funcional con registro, búsqueda, filtros y dashboard. Desplegable con Docker.
- Fase 2 — Multi-usuario corporativo**
Autenticación SSO (Azure AD), PostgreSQL, despliegue en Cloud Run, roles y permisos.
- Fase 3 — Integración con IA (Kratos/Orion)**
El agente Kratos consulta el histórico de KukaLog para sugerir soluciones. Registro de incidencias por lenguaje natural vía Teams.

Coste e Infraestructura

Inversión mínima

- **Hardware adicional:** Ninguno (se despliega en infraestructura GCP existente)
- **Licencias:** Ninguna (stack 100 % open source)
- **Cloud Run:** Coste estimado <20€/mes para uso interno
- **Base de datos:** Cloud SQL PostgreSQL — <15€/mes
- **Tiempo de desarrollo Fase 2:** 2–3 semanas
- **Tiempo de desarrollo Fase 3:** 1–2 semanas (infraestructura Orion ya existente)

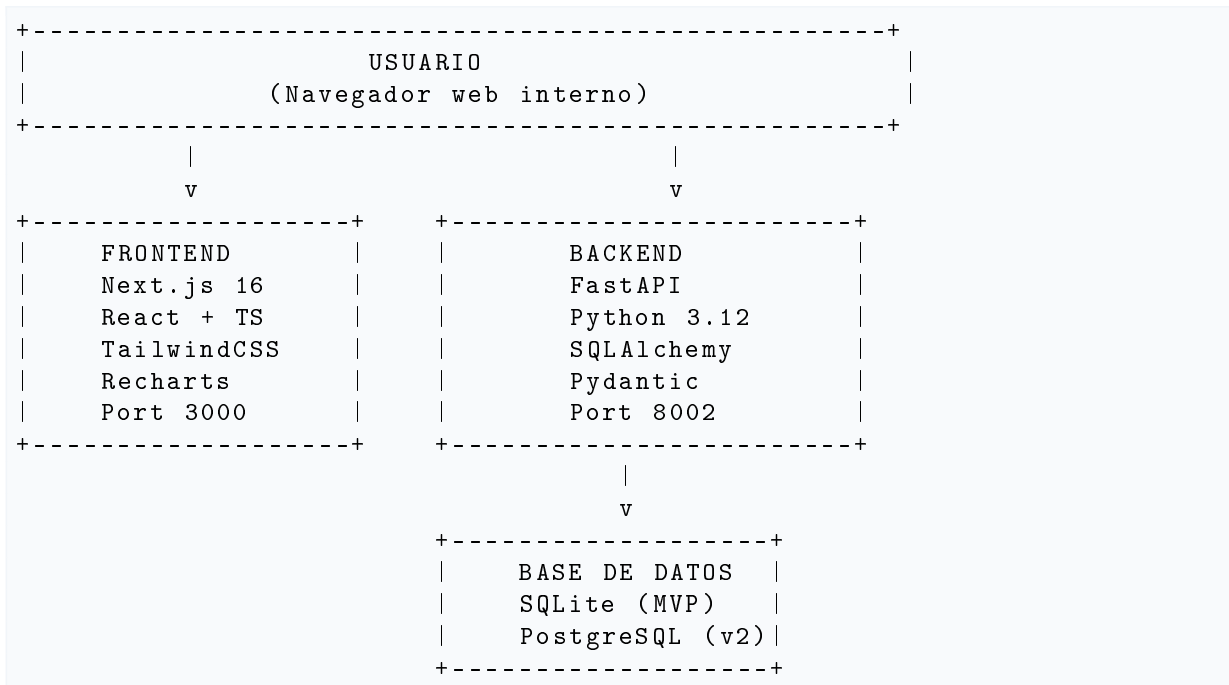
Usuarios Objetivo

- Técnicos de mantenimiento de robots
- Ingenieros de manufactura
- Ingenieros de programación de robots
- Jefes de turno

- Ingenieros de proyecto

Parte II — Documentación Técnica

Arquitectura del Sistema



Flujo de datos

1. El usuario accede al frontend vía navegador
2. El frontend hace peticiones REST al backend (FastAPI)
3. El backend ejecuta queries contra la base de datos
4. Los resultados se devuelven en JSON al frontend
5. El frontend renderiza la interfaz con los datos

Stack Tecnológico

Componente	Tecnología
Backend	Python 3.12 + FastAPI + Uvicorn
Frontend	Next.js 16 + React 19 + TypeScript
Estilos	TailwindCSS v4
Gráficas	Recharts
Iconos	Lucide React
Base de datos (MVP)	SQLite
Base de datos (producción)	PostgreSQL
Contenedorización	Docker + Docker Compose
Cloud	Google Cloud Platform (Cloud Run)

Modelo de Datos

```

CREATE TABLE incidents (
  id          INTEGER PRIMARY KEY,
  created_at  DATETIME,
  updated_at  DATETIME,
  -- Informacion General
  operator    TEXT,
  engineer    TEXT,
  shift       TEXT,          -- manana/tarde/noche
  -- Maquina
  robot_type  TEXT,          -- ej: "6Y 110R1"
  controller  TEXT,          -- KRC2
  cell        TEXT,          -- ej: "Celda-6Y-110"
  line        TEXT,          -- ej: "Linea 6Y"
  -- Incidencia
  error_code  TEXT,          -- ej: "KCP_BLACK", "KSD_NOK"
  category    TEXT,          -- pantalla/hardware/comunicacion...
  priority    TEXT,          -- critica/alta/media/baja
  status      TEXT,          -- abierta/en_progreso/cerrada
  title       TEXT,
  description  TEXT,
  symptoms    TEXT,
  -- Solucion
  solution    TEXT,
  steps       TEXT,
  downtime_min INTEGER,
  result      TEXT,          -- resuelto/parcial/pendiente
  -- Extra
  notes       TEXT,
  images      TEXT           -- JSON array de URLs
);

```

Categorías de incidencia

- pantalla — KCP en negro, congelada
- hardware — PC NOK, condensadores
- seguridades — ESC-CI, modo operación
- comunicación — ServoBus, watchdog, IBS
- accionamiento — KSD, ejes sin tensión
- desincronismo — Pérdida ejes, baterías
- ventilación — Turbinas, ventiladores
- alimentación — KPS, fuentes
- software — SSD, KUKACROSS, DCP
- eléctrica — Conectores, cableado

API Reference

Método	Endpoint	Descripción
GET	/api/stats	Métricas para dashboard
GET	/api/incidents	Lista con filtros (query params)
GET	/api/incidents/{id}	Detalle de incidencia
POST	/api/incidents	Crear nueva incidencia
PUT	/api/incidents/{id}	Editar incidencia
POST	/api/seed	Generar datos de prueba

Filtros disponibles (query params)

- search (full-text)
- status
- priority
- category
- controller
- line
- robot_type
- error_code
- operator
- engineer
- cell
- skip / limit

Ejemplo de petición

```
GET /api/incidents?search=Green+Goo&category=hardware&status=cerrada
```

Ejemplo de respuesta (stats)

```
{
  "total": 43,
  "open": 9,
  "closed": 26,
  "in_progress": 8,
  "avg_downtime": 112.3,
  "top_errors": [
    {"error_code": "KCP_BLACK", "count": 7},
    {"error_code": "KSD_NOK", "count": 5}
  ],
  "top_robots": [...],
  "by_category": [...],
  "by_week": [...],
  "recent": [...]
}
```

Frontend — Páginas

Ruta	Funcionalidad
/	Dashboard con KPIs, gráficas, tabla de recientes
/incidents	Lista completa con búsqueda y filtros avanzados
/incidents/[id]	Detalle completo de una incidencia
/incidents/new	Formulario de registro de nueva incidencia
/search	Buscador dedicado con sugerencias rápidas

Diseño visual

- Paleta: azul corporativo Ford (#003478) + naranja KUKA (#FF6600)
- Tipografía: Inter (system font)
- Sidebar fija con navegación
- Cards con bordes y sombras sutiles
- Diseño industrial-corporativo premium

Deployment

Docker Compose (local / desarrollo)

```
services:
  backend:
    build: ./backend
    ports:
      - "8002:8002"
  frontend:
    build: ./frontend
    ports:
      - "3080:3000"
    depends_on:
      - backend
```

Comandos

```
# Levantar todo
docker compose up -d

# Ver logs
docker compose logs -f

# Parar
docker compose down
```

Cloud Run (producción)

```
# Build y push
docker build -t us-central1-docker.pkg.dev/PROJECT/kukalog/backend .
docker push us-central1-docker.pkg.dev/PROJECT/kukalog/backend

# Deploy
gcloud run deploy kukalog-backend \
  --image us-central1-docker.pkg.dev/PROJECT/kukalog/backend \
  --port 8002 --allow-unauthenticated
```

Parte III — Implementaciones Futuras

Fase 2: Multi-usuario Corporativo

Autenticación SSO (Azure AD)

Ford utiliza Microsoft Entra ID (Azure AD) para la gestión de identidades. La integración:

```
# Backend: middleware de autenticacion
from fastapi_auth import SingleTenantAzureAuthorizationCodeBearer

azure_scheme = SingleTenantAzureAuthorizationCodeBearer(
    app_client_id=AZURE_CLIENT_ID,
    tenant_id=FORD_TENANT_ID,
)

@app.get("/api/incidents")
async def list_incidents(user=Depends(azure_scheme)):
    # user.name, user.roles disponibles
    ...
```

Roles propuestos:

- **Operario** — puede crear incidencias y consultar
- **Ingeniero** — puede crear, editar, cerrar incidencias
- **Admin** — gestión completa + dashboard analytics

Migración a PostgreSQL

```
# Solo cambia la connection string
DATABASE_URL = "postgresql://user:pass@host:5432/kukalog"
# SQLAlchemy abstraer el resto - zero cambios en queries
```

Cloud SQL PostgreSQL en GCP: alta disponibilidad, backups automáticos, escalado vertical.

Upload de archivos multimedia

- Fotos de la incidencia (conector dañado, Green Goo visible, pantalla de error)
- Vídeos cortos de síntomas
- PDFs técnicos adjuntos
- Almacenamiento en Google Cloud Storage (GCS)
- Referencia en campo `images` como JSON array de URLs firmadas

Fase 3: Integración con Agentes IA

Kratos consulta KukaLog

Actualmente, Kratos (agente experto en robots KUKA dentro de la plataforma Orion) utiliza una knowledge base estática de 420 líneas. Con la integración:

```
# Nueva tool para el agente Kratos
async def search_kukalog(query: str, filters: dict) -> list:
    """Busca en el historico real de incidencias."""
    response = await httpx.get(
```

```

        "http://kukalog-backend/api/incidents",
        params={"search": query, **filters}
    )
    return response.json()

```

Ejemplo de uso:

Conversación con Kratos vía Teams

Ingeniero: “El robot 6Y 110R4 tiene la KCP en negro, ¿qué hago?”

Kratos: “Basándome en 4 incidencias previas de KCP en negro en robots 6Y:

1. Causa más frecuente: Green Goo en conectores KPS (3/4 casos)
2. Verificar salvapantallas KUKA y Windows
3. Si persiste: comprobar estado de PC (condensadores hinchados)
4. Solución definitiva si recurrente: cambio de armario completo

[Fuentes: Incidencia #7, #12, #28, #34 en KukaLog]”

Registro automático desde lenguaje natural

Operario (via Teams): "El 8XY 170R3 se ha quedado con la maleta en negro otra vez, tenia green goo en los conectores del KPS, lo hemos limpiado y ha vuelto. 20 min de parada."

```

|
v  (Agente IA extrae campos estructurados)

{
  "robot_type": "8XY 170R3",
  "error_code": "KCP_BLACK",
  "category": "pantalla",
  "title": "KCP en negro - Green Goo en conectores KPS",
  "description": "...",
  "solution": "Limpieza de conectores KPS",
  "downtime_min": 20,
  "status": "cerrada",
  "result": "resuelto"
}

|
v  (POST /api/incidents automatico)

```

Incidencia creada en KukaLog sin intervencion manual

Analytics avanzado

Con datos acumulados, se podrían generar:

- **Predicción de fallos** — Robots con patrón de incidencias creciente
- **Alertas proactivas** — “El robot 6Y 50R4 ha tenido 3 incidencias de Green Goo en 2 meses, programar cambio de armario”
- **Análisis de causa raíz** — Correlación entre categorías, líneas y turnos
- **KPIs de mantenimiento** — MTTR, MTBF por robot/línea

- **Comparativa entre líneas** — Qué líneas concentran más fallos y por qué

Fase 4: Multi-planta

- Filtro por planta (Valencia, Saarlouis, Cologne...)
- Compartir knowledge base entre plantas
- Un fallo resuelto en Valencia aparece como sugerencia en Saarlouis
- Dashboard corporativo global

Equipo y Créditos

Rol	Persona
Desarrollador	Miguel Serra Ferrando
Tutor	Javi Llopis

Dato	Valor
Organización	Ford Motor Company
Departamento	Body Construction
Planta	Valencia (Almussafes)
Fecha inicio	Mayo 2026
Estado	MVP completado

Licencia: Documento interno Ford Motor Company — Uso restringido.